



GLITCHLOCK

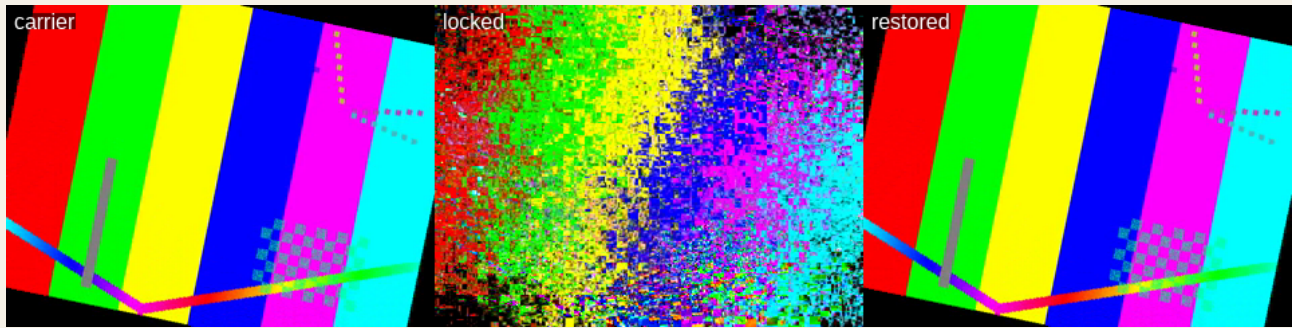
DOC NO. RL-370-A REV. A RIPOSTE LABORATORIES

Reversible, keyed bitstream scrambling for video,
built on FFglitch, and for MP2 audio, in pure Python.

CONTENTS

▶ Why this works at all	3
▶ What does not work, and why it is excluded	5
▶ Install	5
▶ Usage	6
prepare – make a lockable carrier	6
lock	7
unlock	7
verify	8
▶ Public-key recipients	8
▶ Streaming	9
▶ Audio	10
▶ The manifest	11
▶ Self-test	12
▶ Known limitations	13
▶ Status	14
▶ Is this encryption?	14
▶ License	15

You take a video, scramble its guts with a key, and get back a file that still plays – as a mess. Later, with the key and the manifest, you get the original back **byte for byte**. Not “visually close”. Identical SHA-256.



Left: the carrier. Middle: the same file after glitchlock lock, still a valid, playable MPEG-2 stream. Right: after glitchlock unlock – pixel-identical to the left, because the file is bit-identical to the left.

CONSOLE

```
$ glitchlock prepare holiday.mp4 -o carrier.mpg
$ glitchlock lock carrier.mpg -o locked.mpg -m manifest.json --key-file key.bin
locking layer 1/2: mv
    576000/576000 slots across 240 frames
locking layer 2/2: qscale
    7500/7500 slots across 250 frames
self-test: pass (byte-exact)

$ glitchlock unlock locked.mpg -o restored.mpg -m manifest.json --key-file key.bin
integrity: OK - byte-exact match with the original carrier
```

SEC.01

WHY THIS WORKS AT ALL

Datamoshing is normally a one-way trip. You corrupt a bitstream, it looks great, and the original is gone. glitchlock is reversible because it only ever writes values the codec can store **exactly**, and it knows precisely which values those are.

Motion vectors are the key. In MPEG-2 and MPEG-4, a decoded motion vector is sign-extended to a fixed bit width derived from the frame's `f_code`. Write a value outside that width and the decoder does not clamp it or reject it – it **wraps** it,

modulo the range. So each MV component lives in a genuine cyclic group, $\mathbb{Z}_N$, where N is a power of two:

CODEC	WIDTH	N AT F_CODE=1	DOMAIN
MPEG-1/2 video	<code>f_code + 4</code>	32	<code>[-16, 15]</code>
MPEG-4 / H.263	<code>f_code + 5</code>	64	<code>[-32, 31]</code>

In a B-frame the two directions have **different** `f_codes`: forward vectors are coded against `fcode`, backward ones against `bcode`, which FFedit reports as a separate field. On a `-bf 2` MPEG-2 encode, 27 of 32 B-frames had `bcode ≠ fcode`. Using the forward code for both is wrong in both directions – too narrow and a legal file is refused, too wide and a value gets written that the codec cannot store. Measured: writing 17 into a slot whose true `bcode` is 1 read back as -15.

Any bijection on $\mathbb{Z}_N$ is losslessly reversible. glitchlock uses two, both keyed:

- **Substitution** – add a keystream offset to each value, modulo its own domain.
- **Permutation** – shuffle values between slots, but only between slots with the **same** domain, so a value can never land somewhere too small to hold it.

MPEG-2's quantiser scale (`qscale`) is a second carrier: a fixed-width 5-bit field with legal values 1–31, which behaves the same way.

That per-slot domain is not a guess. It is checked against every value in your actual file before anything is written, and a violation aborts the lock rather than silently wrapping a value into oblivion.

SEC.02 WHAT DOES **not** WORK, AND WHY IT IS EXCLUDED

FFedit will happily let you edit DCT coefficients, and they make gorgeous garbage. They are also not reversible, so glitchlock refuses them:

```
$ glitchlock inspect carrier.mpg
q_dct      NOT reversible - DCT coefficients are variable-length coded; edits desynchronise the slice
q_dc       NOT reversible - DC coefficients are differentially coded; edits desynchronise the slice
mv         reversible - glitchlock can use this
qscale     reversible - glitchlock can use this
```

DC and AC coefficients are variable-length coded, so changing a value changes how many bits it occupies, which shifts everything after it. Measured directly: after writing scrambled DC values and re-exporting, the **block structure itself** came back different – the decoder reported `ac-tex damaged`, `invalid cbp` and `slice mismatch`. There is no manifest that can undo that, so the feature is not offered. See `docs/pdf/design.pdf` for the full evidence.

SEC.03 INSTALL

You need FFglitch's binaries (`ffedit`, `ffgac`) on your `PATH`. Grab them from <https://ffglitch.org/download/> – no build required for MPEG-1/2/4 carriers.

CONSOLE

```
$ pip install git+https://github.com/armeehn/glitchlock
$ glitchlock inspect some.mpg
```

H.264 and HEVC carriers need the patched `ffedit` from `ffglitch/`: either build it (`ffglitch/build.sh`, a few minutes, needs `gcc` and `make`) or take the static Linux x86-64 build attached to the `ffglitch-hevc-1` release of this repo, which is what CI runs. The core is `stdlib-only`; `-recipient` needs `pip install "glitchlock[recipients]"`.

If FFglitch lives somewhere unusual, point at it with `GLITCHLOCK_FFGLITCH_HOME=/opt/ffglitch`.

SEC.04

USAGE

PREPARE – MAKE A LOCKABLE CARRIER

Stock FFglitch exposes nothing reversible for H.264/HEVC. `prepare` transcodes to a glitchable MPEG-2 or MPEG-4 elementary stream using the standard FFglitch encoder recipe (`+nopimb+forcemv`, so every macroblock carries a real vector), or to H.264 or HEVC when the patched `ffedit` from `ffglitch/` is on PATH.

CONSOLE

```
$ glitchlock prepare input.mkv -o carrier.mpg --codec mpeg2video --qscale 6 --gop 25
$ glitchlock prepare input.mkv -o carrier.264 --codec h264 --gop 25 --closed-gop
$ glitchlock prepare input.mkv -o carrier.265 --codec hevc --gop 25 --closed-gop
```

The H.264 carrier is Main profile with CAVLC entropy coding and two B-frames, made by the system `ffmpeg`'s `libx264` (`-qscale` does not apply; it uses CRF 23). CAVLC matters: motion vector differences are plain Exp-Golomb codes there, so an edited vector never changes the shape of the bitstream. CABAC streams are refused, not corrupted. Scrambled vectors stay within ± 512 px, the Level 3.1 vertical range, so hardware decoders still play the result. Building the patched FFglitch: `ffglitch/build.sh /tmp/ffg /opt/ffglitch-h264` (see `ffglitch/NOTES.md`).

The HEVC carrier is Main (or Main 10) with two B-frames from `libx265`, CRF 23, without wavefront parallel processing. HEVC has no CAVLC, so an edited vector cannot be dropped into the bitstream: the patched `ffedit` logs every CABAC bin the decoder reads, swaps the bins of each scrambled vector and re-encodes the slice. Unlocking re-encodes it back to x265's exact bytes. Streams with WPP, tiles or PCM are refused, not corrupted.

This step is lossy and drops audio – it is a transcode. **The carrier is the plaintext.** Everything after this point is bit-exact.

LOCK

CONSOLE

```
$ glitchlock lock carrier.mpg -o locked.mpg -m manifest.json --key-file key.bin
```

FLAG	EFFECT
<code>--features mv,qscale</code>	which carriers to use (default: all verified ones present)
<code>-mode full</code>	full (substitute + permute), substitute, or permute
<code>--intensity 0.3</code>	scramble only a key-selected 30% of slots – a dial, not a weakening of reversibility
<code>-keyless</code>	store the seed in the manifest; the manifest alone can unwind it
<code>-password / -key-file</code>	key material; scrypt is used for passphrases
<code>-no-selftest</code>	skip the reversibility proof (not recommended)

`--mode permute` is worth knowing about: it moves vectors around without changing their values, so the frame keeps its exact motion histogram and the result looks like the scene tearing itself apart rather than dissolving into noise.

UNLOCK

CONSOLE

```
$ glitchlock unlock locked.mpg -o restored.mpg -m manifest.json --key-file key.bin
```

Exits non-zero if the restored file does not match the carrier digest recorded in the manifest, so it is safe to use in a script.

VERIFY

Lock and unlock a file in a scratch directory and report whether the round trip was exact. Useful for checking a new codec or a new FFglitch release.

CONSOLE

```
$ glitchlock verify carrier.mpg
round trip: EXACT
```

SEC.05

PUBLIC-KEY RECIPIENTS

Lock a file for someone who has published a public key, with no shared secret:

CONSOLE

```
$ glitchlock keygen -o me.key
identity: me.key (mode 0600 - keep it secret)
public key: glk-pub-v1:FAekY8ufscY1elddq0gee7hUPBxCcuPq3TnrCSt1Bko=
fingerprint: ba688596449807df

$ glitchlock lock carrier.mpg -o locked.mpg -m manifest.json \
  --recipient glk-pub-v1:FAekY8ufscY1elddq0gee7hUPBxCcuPq3TnrCSt1Bko=
recipients: 1 (ba688596449807df)

$ glitchlock unlock locked.mpg -o restored.mpg -m manifest.json --identity me.key
integrity: OK - byte-exact match with the original carrier
```

The asymmetric key never touches the video. This is ordinary hybrid encryption: a random 32-byte content key drives the scrambler, and that content key is sealed to each recipient with X25519 → HKDF-SHA256 → ChaCha20-Poly1305. The manifest carries the sealed copies and nothing else; opening one needs the matching private key. Repeat `--recipient` to lock for several people at once.

Needs the `cryptography` package — `pip install 'glitchlock[recipients]'`. The core scrambler stays dependency-free.

This solves key distribution, not information leakage. The locked video is still a playable video whose residual picture survives. Public keys make it practical to hand a locked file to someone; they do not make the ciphertext safe to publish. [docs/pdf/security.pdf](#) is explicit about this.

SEC.06

STREAMING

Yes, it works on a stream, at GOP granularity, and the round trip stays byte-exact. Encode with closed GOPs, split on sequence headers, and give every segment its own number:

CONSOLE

```
$ glitchlock prepare live.mkv -o carrier.mpg --gop 12 --closed-gop
$ glitchlock lock seg_0007.mpg -o locked_0007.mpg -m /dev/null --segment 7 --key-file key.bin
```

The segment number matters more than it looks: without it, every segment's frame 0 derives an identical plan from the same key and nonce, and the whole broadcast shares one keystream. The round trip still works, which is what makes it dangerous.

The receiver finds segment boundaries by scanning the locked stream itself, and can synthesise its manifest from session parameters – so nothing has to be sent alongside the video. Latency is one GOP (480 ms at GOP 12/25 fps). 720p keeps up with a live feed on 4 cores; 1080p needs more.

`stream-lock` and `stream-unlock` do this over pipes, so the sender and the receiver are one command each. The receiver needs the key and a small session record (nonce, codec, features, first segment number); no manifest travels:

CONSOLE

```
$ glitchlock prepare live.mkv -o carrier.m4v --codec mpeg4 --closed-gop
$ glitchlock stream-lock -i carrier.m4v --key-file key.bin --session s.json --gops 5 | nc host 9000
$ nc -l 9000 | glitchlock stream-unlock --session s.json --key-file key.bin | ffmpeg -f m4v -
```

Segments are locked by a worker pool but written strictly in order, and at most twice `-workers` are in flight, so memory stays bounded. A stream that ends inside a GOP fails on its last piece only; everything before it is out.

See `docs/pdf/streaming.pdf` for the measurements and for what is still missing – there is no TS/HLS wrapping and no daemon yet.

SEC.07

AUDIO

MPEG-1/2 Audio Layer II (MP2) elementary streams lock the same way, with the same manifest, MAC, self-test and no-op refusal. No FFglitch involved: the frame parser and writer are pure Python (`glitchlock/mp2.py`).

CONSOLE

```
$ ffmpeg -i song.flac -c:a mp2 -b:a 192k song.mp2
$ glitchlock inspect song.mp2
codec:      mp2 (MPEG-1 Audio Layer II)
stream:     48000 Hz, stereo, 192 kbps, 8843 frames (212.2 s)
lockable features: samples, scalefactors
$ glitchlock lock song.mp2 -o locked.mp2 -m m.json --key-file key.bin
$ glitchlock unlock locked.mp2 -o back.mp2 -m m.json --key-file key.bin
integrity: OK - byte-exact match with the original carrier
```

Why MP2 and not MP3 or AAC: Layer II has no entropy coding. After the header, bit allocation and scfsi, a frame is nothing but fixed-width fields – 6-bit scale-factor indices and quantised subband samples whose width the allocation dictates (3, 5 and 9 levels pack three samples into one 5-, 7- or 10-bit codeword). Every such field gets $(v + k) \bmod \text{range}$, k from the keystream, with `range` the field's **legal** set: `nlevels` for a plain sample, `nlevels*3` for a grouped codeword, 63 for a scalefactor (index 63 is reserved). XOR would be wrong here, because it can produce a value outside that set. The result parses in every decoder, decodes with the right number of samples, sounds like shaped noise (measured $|r| < 0.03$ against the original across the fixture matrix) and inverts exactly. `--mode permute` shuffles field values within a frame between fields of the same range; `full` does both.

Header, CRC, bit allocation and scfsi are never touched: they define the geometry. That is also why a protected stream keeps a valid CRC-16 – per ISO/IEC 11172-3 2.4.3.1 (and ffmpeg’s `handle_crc`) it covers only header bytes 2-3, the allocation and the scfsi bits, verified with `ffmpeg -err_detect crccheck` on locked frames.

Per-frame keying includes `-segment` exactly as for video, so a stream cut into pieces can be locked piecewise and reassembled. `mp2.FrameReader` / `mp2.iter_mp2_frames` frame a live stream incrementally for a receiver that unlocks frame by frame. Framing accepts every valid header as a frame with no “does the next header chain” check, on purpose: a rule that peeked into frame bodies could decide differently on the ciphertext. Junk between frames (ID3 tags, garbage) passes through verbatim.

Supported: MPEG-1 (32/44.1/48 kHz) and MPEG-2 LSF (16/22.05/24 kHz), mono, stereo, dual channel and joint stereo (intensity bound honoured), all five allocation tables, with or without CRC. Not supported: Layer I, Layer III, AAC, free-format bitrate, and MP2 muxed inside a container (demux to an elementary stream first). Detection sniffs the sync word and two chained headers, not the extension. Throughput is about 300 frames/s on one core for 192 kbps stereo, roughly 7x real time.

SEC.08

THE MANIFEST

The manifest is the record of how the file was scrambled. It is small – 1,085 bytes for a 3.1 MB video – because it stores **parameters**, not data:

JSON

```
{
  "format": "glitchlock-manifest",
  "codec": "mpeg2video",
  "nonce": "273783ef4fc8ba9522477d549346e880",
  "carrier_sha256": "8d5aacc6...",
  "locked_sha256": "11101a6d...",
  "layers": [
    { "feature": "mv", "mode": "full", "intensity": 1.0,
      "frames": 240, "slots_total": 576000, "slots_touched": 576000,
```

```
    "repairs": {} }  
  ],  
  "selftest": "pass",  
  "mac": "b08623b5..."  
}
```

It never contains the key. It is authenticated with HMAC-SHA256 under your key, so a wrong passphrase or an edited manifest is refused up front instead of quietly producing rubbish.

`repairs` is the honesty valve. After locking, `glitchlock` reads its own output back and compares it against what it intended to write. Anything the encoder did not reproduce exactly is recorded here as a literal original value, so recovery stays exact regardless. On `mv` and `qscale` this list comes back empty – which is the expected result, and is asserted by the test suite rather than assumed.

SEC.09

SELF-TEST

By default `lock` unlocks its own output in a temp directory and compares the digest to the input before it writes the manifest. If it does not match, **no manifest is written and the command fails**. A lock that cannot be proven reversible is not shipped.

Reversibility is not the only thing worth proving, though. A byte-identical copy of the carrier unlocks to itself perfectly, so the self-test alone will wave it through. `lock` therefore also refuses to write output that is byte-identical to its input, because that output **is** the plaintext:

CONSOLE

```
$ glitchlock lock all-intra.m4v -o locked.m4v -m m.json --key-file key.bin  
glitchlock: the locked file is byte-identical to the carrier: nothing was  
actually scrambled, so this output is the plaintext. ...
```

Three ordinary things land here, and none of them used to say a word:

-
- ▶ an all-intra carrier (`-gop 1`) has no motion vectors to scramble;
 - ▶ a static shot with `--mode permute` – every vector is `(0,0)`, and permuting values that are all equal is the identity map. Measured on a solid-grey clip: 29,550 slots reported scrambled, output identical;
 - ▶ a very low `-intensity` on a small file can select zero slots.

Pass `-allow-noop` if you genuinely want a copy.

SEC.10

KNOWN LIMITATIONS

- ▶ **Interlaced carriers do not work.** An interlaced MPEG-2 carrier (`-flags +ilme+ildct`) fails with a geometry error on every nonce; an interlaced MPEG-4 one fails on **some** nonces – measured 5 failures in 12. Field pictures split motion vectors per field and the re-exported slot layout does not match what was written. Deinterlace before `prepare`.
- ▶ **verify results are per-nonce.** Because of the above, a single passing run is weak evidence for a carrier **class**. `verify` now derives its nonce from the run index so a result is reproducible, and `-repeat N` sweeps N of them: `console $ glitchlock verify interlaced.m4v --repeat 12 ... result: EXACT=7, FAILED=5` this carrier is NOT reliably lockable: the outcome depends on the nonce, so one passing run proves nothing
- ▶ **FFglitch 0.10.2 aborts on some B-heavy MPEG-2 files.** `ffedit -f qscale` on a `-bf 4` carrier dies with `free(): chunks in smallbin corrupted (exit 134)` before `glitchlock` sees any data. It is an upstream crash, not a `glitchlock` one, and it fails closed – but it means such files cannot be locked with the `qscale` layer. `-features mv` still works on them.

SEC.11

STATUS

Verified on FFglitch 0.10.2 for MPEG-2, MPEG-4 part 2, H.264 (CAVLC) and HEVC, and on MP2 audio against ffmpeg 8.1. 259 tests pass, including byte-exact round trips through real bitstreams for every mode, all four video codecs, B-frames, 4MV, qpel, a chunked stream, and 20 MP2 fixtures across rates, bitrates, modes and CRC. Beyond the suite, a 900-configuration MPEG sweep across 18 source clips (odd dimensions, 16x16, single frame, 60 fps, greyscale, pure noise, 720p) and a 20-configuration x265 sweep (CTB 16/32/64, B-pyramid, AMP, weighted prediction, lossless CUs, Main 10, 720p) round-tripped byte-exact in every case that scrambled anything at all.

Lockable: `mpeg1video`, `mpeg2video` and `mpeg4` with stock FFglitch; `h264` and `hevc` with the patched build. The domain table also lists H.263, MSMPEG-4 v1-v3, WMV1/2 and FLV1 because their motion vector geometry is known, but FFglitch 0.10.2 exposes no editable features for any of them – `ffedit -i` lists nothing, so there is nothing to scramble. They are kept in the table against a future FFglitch that does expose them.

SEC.12

IS THIS ENCRYPTION?

It is a keyed, reversible cipher, and the key really is required. It is **not** a replacement for encrypting a file, and you should read `docs/pdf/security.pdf` before treating it as one. The short version: the ciphertext is a video, and a video that still decodes leaks information about itself. Use it for reversible glitch art, for obfuscation, for watermarking experiments – not for protecting a secret. If you want confidentiality, use age or GPG.

SEC.13

LICENSE

MIT — see `LICENSE`. FFglitch itself is GPL and is used here as an external binary, not linked. The patches under `ffglitch/` modify FFmpeg code and stay under FFmpeg's LGPL-2.1+; binaries built from them with `build.sh` (including the release asset CI uses) are GPL-2.0+, and their source is the FFglitch 0.10.2 tarball plus those patches.

SOURCE README.md SHA 9350c8fd6186 BUILT 2026-09-13